UNIVERSIDADE DE MOGI DAS CRUZES
SISTEMAS DE INFORMAÇÃO

DANIEL CARDOSO LEÃO

DASHBOARD ADMINISTRATIVO DO SISTEMA DE CHAMADOS THINDESK:

documentação técnica

MOGI DAS CRUZES
2026

DANIEL CARDOSO LEÃO

DASHBOARD ADMINISTRATIVO DO SISTEMA DE CHAMADOS THINDESK:

documentação técnica

Documentação técnica apresentada à disciplina de Aplicativos Web do curso de Sistemas de Informação da Universidade de Mogi das Cruzes, como requisito parcial para avaliação.

Professor: Prof. Alessandro Aparecido da Silva Horas.

MOGI DAS CRUZES

2026

RESUMO

Este documento descreve a análise, o projeto e a implementação do dashboard administrativo do Thindesk, um sistema de chamados técnicos de TI. O painel permite que o responsável pelos chamados acompanhe o andamento da operação, a situação dos chamados e as considerações técnicas registradas, e identifique dados relevantes para a tomada de decisão, como backlog, atrasos em relação ao acordo de nível de serviço (SLA), tipos de chamado mais demandados e desempenho da equipe. A solução foi construída com Java 21, Spring Boot 4, Spring Security, Thymeleaf e MongoDB Atlas, reaproveitando a base de login seguro desenvolvida em atividade anterior. Todos os dados do painel vêm de uma API REST projetada neste trabalho, com 13 endpoints versionados, filtros combináveis, erros padronizados segundo a RFC 9457 e documentação OpenAPI. O documento apresenta os requisitos funcionais e não funcionais, a arquitetura, o modelo de dados, as regras de negócio, a documentação completa dos endpoints, as telas do dashboard, as decisões de projeto e as evidências de testes: 75 testes automatizados e 40 casos de integração executados contra o banco real, todos aprovados.

Palavras-chave: Dashboard. API REST. Spring Boot. MongoDB Atlas. Chamados técnicos.

ABSTRACT

This document describes the analysis, design and implementation of the administrative dashboard of Thindesk, an IT help desk ticketing system. The dashboard lets the person in charge of the tickets follow the operation, the status of each ticket and the technical notes recorded, and identify relevant data for decision making, such as backlog, service level agreement (SLA) breaches, most requested ticket types and team performance. The solution was built with Java 21, Spring Boot 4, Spring Security, Thymeleaf and MongoDB Atlas, reusing the secure login foundation developed in a previous assignment. All dashboard data comes from a REST API designed in this work, with 13 versioned endpoints, combinable filters, errors standardized according to RFC 9457 and OpenAPI documentation. The document presents the functional and non-functional requirements, the architecture, the data model, the business rules, the complete endpoint documentation, the dashboard screens, the design decisions and the test evidence: 75 automated tests and 40 integration cases run against the real database, all passing.

Keywords: *Dashboard. REST API. Spring Boot. MongoDB Atlas. Help desk tickets.*

LISTA DE FIGURAS

Figura 1 - Arquitetura em camadas do dashboard Thindesk 16
Figura 2 - Modelo de dados das coleções do MongoDB 18
Figura 3 - Fluxo de situações do chamado 20
Figura 4 - Filtros e indicadores gerais do dashboard 30
Figura 5 - Chamados abertos e atendidos por semana, com filtros aplicados 31
Figura 6 - Situação dos chamados e backlog por prioridade 31
Figura 7 - Ranking dos tipos de chamados mais atendidos 32
Figura 8 - Desempenho por técnico 32
Figura 9 - Lista de chamados 33
Figura 10 - Detalhe do chamado com considerações técnicas e ações 34
Figura 11 - Documentação interativa da API (Swagger UI) 35
Figura 12 - Dashboard em tela de celular 36

LISTA DE QUADROS

Quadro 1 - Requisitos funcionais e forma de atendimento 12
Quadro 2 - Requisitos não funcionais 13
Quadro 3 - Tecnologias utilizadas no projeto 14
Quadro 4 - Pacotes do projeto 17
Quadro 5 - Exemplo de documento da coleção tickets 19
Quadro 6 - Prazo de solução (SLA) por prioridade 19
Quadro 7 - Definições usadas nos indicadores 20
Quadro 8 - Branches do repositório 21
Quadro 9 - Formas de autenticação da API 22
Quadro 10 - Filtros comuns (RF05) 23
Quadro 11 - Exemplo de resposta de erro de validação 23
Quadro 12 - Códigos de erro da API 24
Quadro 13 - Endpoints da API (prefixo /api/v1) 24
Quadro 14 - Requisição e resposta de GET /dashboard/summary 25
Quadro 15 - Parâmetro específico de GET /dashboard/timeline 25
Quadro 16 - Requisição e resposta resumida de GET /dashboard/timeline 26
Quadro 17 - Resposta resumida de GET /dashboard/status 26
Quadro 18 - Parâmetros de ranking 27
Quadro 19 - Resposta resumida de GET /dashboard/types?limit=2 27
Quadro 20 - Resposta resumida de GET /dashboard/filters 27
Quadro 21 - Parâmetros de paginação de GET /tickets 28
Quadro 22 - Resposta resumida de GET /tickets/{id} 28
Quadro 23 - Campos de POST /tickets 28
Quadro 24 - Requisição de POST /tickets 29
Quadro 25 - Erros de regra de negócio (HTTP 422) 29
Quadro 26 - Decisões de projeto e justificativas 37

Quadro 27 - Testes automatizados 38
Quadro 28 - Testes de integração da API contra o banco real 38
Quadro 29 - Testes manuais pela interface 40
Quadro 30 - Variáveis de configuração (.env) 41

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
API	Application Programming Interface
CSP	Content Security Policy
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DTO	Data Transfer Object
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
ISO	International Organization for Standardization
JDK	Java Development Kit
JSON	JavaScript Object Notation
MVC	Model-View-Controller
NBR	Norma Brasileira
REST	Representational State Transfer
RF	Requisito Funcional
RFC	Request for Comments
RNF	Requisito Não Funcional
SLA	Service Level Agreement (acordo de nível de serviço)
TI	Tecnologia da Informação
TLS	Transport Layer Security
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UTC	Coordinated Universal Time

SUMÁRIO

1 INTRODUÇÃO 10
1.1 Objetivo geral 10
1.2 Objetivos específicos 10
1.3 Organização do documento 11
2 REQUISITOS 12
2.1 Requisitos funcionais 12
2.2 Requisitos não funcionais 12
3 TECNOLOGIAS UTILIZADAS 14
4 ANÁLISE E PROJETO 15
4.1 Arquitetura 15
4.2 Modelo de dados 17
4.3 Regras de negócio 19
4.4 Versionamento com Gitflow 20
5 API REST 22
5.1 Autenticação e autorização 22
5.2 Convenções e filtros 22
5.3 Tratamento de erros 23
5.4 Visão geral dos endpoints 24
5.5 Indicadores gerais 25
5.6 Série histórica 25
5.7 Situação dos chamados 26
5.8 Tipos mais atendidos e desempenho por técnico 26
5.9 Opções dos filtros 27
5.10 Consulta de chamados 27
5.11 Atualização de chamados 28
6 DASHBOARD ADMINISTRATIVO 30

7 PRINCIPAIS DECISÕES DE PROJETO 37
8 TESTES E VALIDAÇÃO 38
8.1 Testes automatizados 38
8.2 Testes de integração 38
8.3 Testes manuais 39
9 EXECUÇÃO DO PROJETO 41
10 CONCLUSÃO 42
REFERÊNCIAS 43

1 INTRODUÇÃO

Em um setor de tecnologia da informação, os chamados técnicos são a principal forma de registrar e acompanhar os problemas e as solicitações dos usuários. À medida que o volume cresce, o responsável pela equipe precisa de uma visão consolidada para saber quantos chamados estão em aberto, quais já passaram do prazo, que tipos de problema são mais frequentes e como cada técnico está atendendo a demanda.

Este trabalho apresenta o dashboard administrativo do **Thindesk**, um sistema de chamados técnicos. O painel consome exclusivamente uma API REST projetada e implementada neste trabalho, integrada a um banco de dados MongoDB Atlas, e reaproveita a base de login seguro desenvolvida na atividade anterior da disciplina, com a mesma pilha tecnológica: Java, Spring Boot, Thymeleaf e MongoDB Atlas.

O enunciado da atividade cita, no objetivo, um sistema de pedidos de lanchonete ou pizzaria, mas a contextualização e todos os requisitos funcionais (RF01 a RF07) tratam de chamados técnicos do Thindesk. A solução segue os requisitos funcionais, ou seja, o domínio de chamados técnicos.

1.1 Objetivo geral

Analisar os requisitos, projetar e implementar um dashboard administrativo integrado ao sistema de chamados Thindesk, alimentado por uma API REST própria e conectada a um banco de dados, que auxilie o responsável pelos chamados na tomada de decisões.

1.2 Objetivos específicos

a) levantar os requisitos funcionais e não funcionais do painel;

b) modelar os dados de chamados, técnicos e considerações técnicas no MongoDB;

c) projetar e implementar os endpoints que alimentam indicadores, gráficos, rankings e filtros;

d) construir as telas do dashboard consumindo a API, com atualização automática dos dados;

e) tratar e validar os dados de entrada e padronizar as respostas de erro;

f) documentar os endpoints com parâmetros, exemplos de requisição e resposta e possíveis erros;

g) comprovar o funcionamento com testes automatizados, testes de integração e testes manuais.

1.3 Organização do documento

A seção 2 apresenta os requisitos. A seção 3 descreve as tecnologias utilizadas. A seção 4 trata da análise e do projeto: arquitetura, modelo de dados, regras de negócio e versionamento. A seção 5 documenta a API REST. A seção 6 apresenta as telas do dashboard. A seção 7 reúne as principais decisões de projeto. A seção 8 traz as evidências de testes, a seção 9 explica como executar o projeto e a seção 10 apresenta a conclusão.

A apresentação deste trabalho segue a NBR 14724 (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011), e as referências seguem a NBR 6023 (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2018).

2 REQUISITOS

Os requisitos foram levantados a partir do enunciado da atividade e organizados em funcionais, que descrevem o que o sistema faz, e não funcionais, que descrevem como ele deve se comportar.

2.1 Requisitos funcionais

O Quadro 1 relaciona cada requisito funcional do enunciado à forma como foi atendido, ao endpoint da API e à tela do dashboard correspondentes.

Quadro 1 - Requisitos funcionais e forma de atendimento

RF	Requisito	Como foi atendido	Endpoint / Tela
RF01	Indicadores gerais de chamados técnicos	Oito indicadores: chamados no período, em aberto (backlog), atendidos, atrasados, taxa de resolução, SLA cumprido, tempo médio de primeira resposta e de solução (com mediana). Comparação com o período anterior e quadro de destaques em texto.	GET /dashboard/summary; cards e destaques
RF02	Análise de chamados atendidos por período (gráficos)	Gráfico combinado de barras (abertos e atendidos) e linha (tempo médio de solução), agrupado por dia, semana ou mês, com escolha automática pelo tamanho do período.	GET /dashboard/timeline; gráfico
RF03	Situação dos chamados	Gráfico de rosca com as seis situações, backlog por prioridade com atrasados, lista dos chamados que exigem atenção, lista completa e detalhe com considerações técnicas e histórico.	GET /dashboard/status, /tickets, /tickets/{id}
RF04	Tipos de chamados mais atendidos	Ranking por tipo com atendidos, total, participação, tempo médio e SLA, alternável entre atendidos e abertos. Complemento: desempenho por técnico.	GET /dashboard/types, /dashboard/technicians
RF05	Filtros de consulta	Período com atalhos, situação, tipo, prioridade, setor e técnico, aplicados a todos os blocos. Na lista: busca textual, só atrasados, ordenação e paginação. Filtros ficam na URL.	GET /dashboard/filters e parâmetros comuns
RF06	Integração com a API	As telas montam a estrutura em Thymeleaf e buscam todos os dados na API via fetch. A API também atende clientes externos com HTTP Basic e é documentada em OpenAPI.	/api/v1/**; todas as telas
RF07	Atualização e tratamento de dados	Atualização automática a cada 60 segundos, botão de atualizar e horário da última carga; abertura, mudança de situação, atribuição e notas pela API. Validação, regras de negócio, normalização e erros padronizados exibidos por bloco.	POST e PATCH /tickets/**

Fonte: Elaborado pelo autor (2026).

2.2 Requisitos não funcionais

O Quadro 2 apresenta os requisitos não funcionais definidos para a solução e como cada um foi atendido.

Quadro 2 - Requisitos não funcionais

RNF	Requisito	Como foi atendido
RNF01	Segurança de acesso	Dashboard e API restritos aos perfis MANAGER e ADMIN por rota e por método. Senhas com BCrypt, proteção CSRF nas requisições com sessão, cabeçalhos CSP, X-Frame-Options e Referrer-Policy, cookie HttpOnly e SameSite.
RNF02	Padronização da API	REST versionada (/api/v1), JSON, códigos HTTP semânticos (200, 201 com Location, 400, 401, 403, 404, 405, 409, 415, 422) e erros no padrão Problem Details (RFC 9457).
RNF03	Desempenho	Filtros executados no MongoDB com índices; projeção sem campos longos nas consultas do dashboard; blocos do painel carregados em paralelo.
RNF04	Confiabilidade dos dados	Controle otimista de versão contra atualizações concorrentes, protocolo gerado por contador atômico e transições validadas no domínio.
RNF05	Usabilidade e responsividade	Layout responsivo testado em 390 px sem rolagem horizontal, estados de carregamento, vazio e erro por bloco, mensagens em português.
RNF06	Manutenibilidade	Camadas separadas, cálculo dos indicadores em funções puras e tema visual definido por variáveis CSS.
RNF07	Testabilidade	75 testes automatizados independentes do banco e roteiro de 40 casos de integração contra o banco real.
RNF08	Documentação	OpenAPI 3 com Swagger UI gerado a partir do código, este documento e coleção Postman.
RNF09	Configuração externa	Credenciais em variáveis de ambiente ou arquivo .env fora do controle de versão; fuso horário, intervalo de atualização e massa de dados configuráveis.

Fonte: Elaborado pelo autor (2026).

3 TECNOLOGIAS UTILIZADAS

A solução usa a mesma pilha tecnológica da atividade de login, cuja base foi reaproveitada como ponto de partida. O Spring Boot fornece configuração automática e servidor embarcado (SPRING, 2026a); o Spring Security cuida da autenticação, da autorização e da proteção contra CSRF (SPRING, 2026b); e o Spring Data MongoDB faz o mapeamento dos documentos e as consultas (SPRING, 2026c). O Quadro 3 lista as tecnologias e o papel de cada uma.

Quadro 3 - Tecnologias utilizadas no projeto

Tecnologia	Versão	Papel no sistema
Java (Eclipse Temurin)	21 LTS	Linguagem e plataforma de execução
Spring Boot	4.0.8	Configuração automática, servidor embarcado e gestão de dependências
Spring MVC e Bean Validation	7.0	Controllers REST e de página, validação de parâmetros e corpos
Spring Security	7.0	Login por formulário, HTTP Basic na API, perfis, CSRF e cabeçalhos
Spring Data MongoDB	5.0	Repositórios, consultas dinâmicas e paginação
MongoDB Atlas e MongoDB Spring Session	4.0	Banco de dados em nuvem e sessões HTTP persistidas
Thymeleaf e Layout Dialect	3.1 / 3.4	Páginas renderizadas no servidor com layout reutilizável
Chart.js (WebJar)	4.5.1	Gráficos do dashboard
springdoc-openapi	3.0.3	Documentação OpenAPI 3 e Swagger UI
JUnit 5, AssertJ, Mockito, MockMvc	-	Testes automatizados
Maven Wrapper	3.9	Build sem instalação prévia do Maven

Fonte: Elaborado pelo autor (2026).

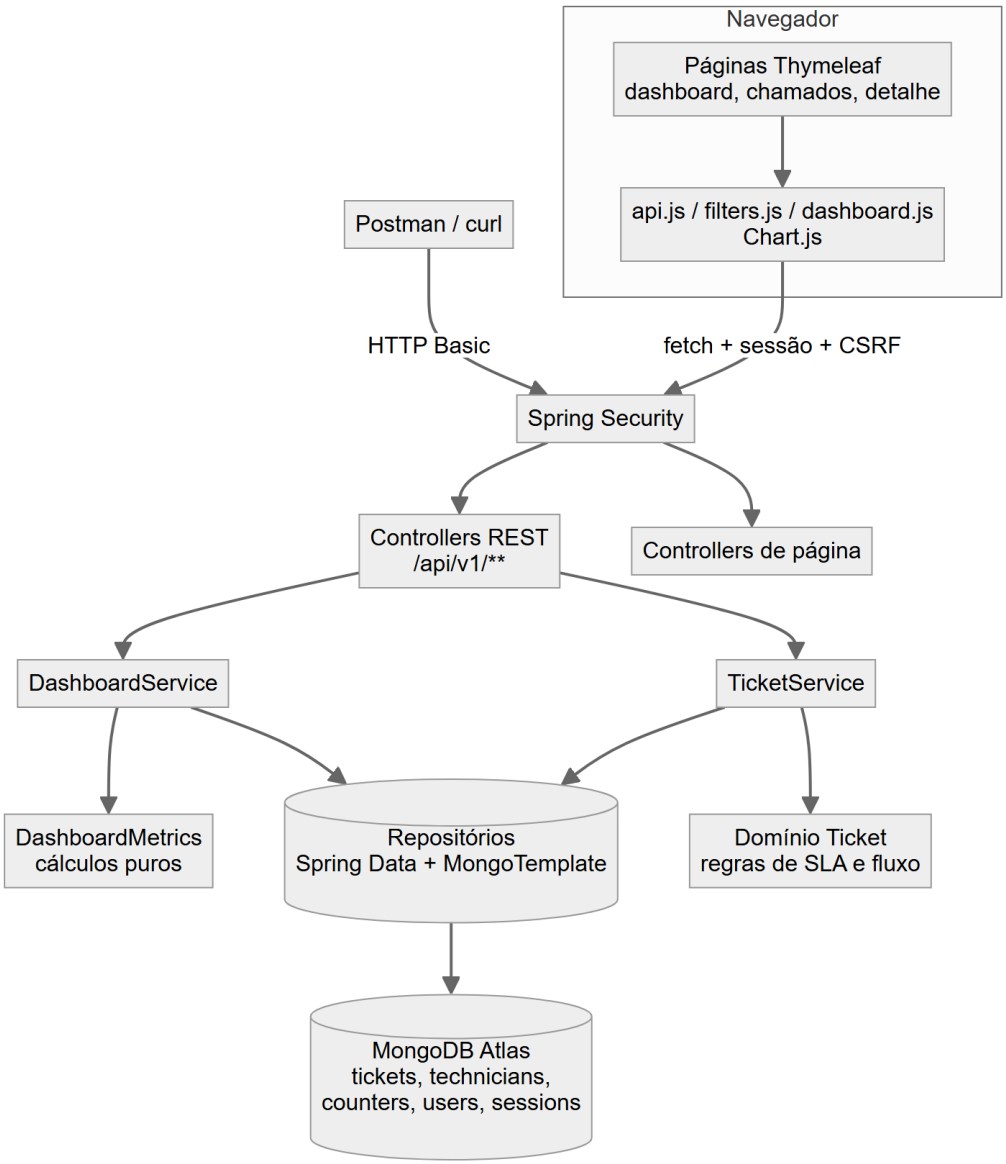
4 ANÁLISE E PROJETO

Esta seção descreve como a solução foi organizada: as camadas da aplicação, o modelo de dados no MongoDB, as regras de negócio dos chamados e o fluxo de versionamento do código.

4.1 Arquitetura

A aplicação segue uma arquitetura em camadas. O navegador carrega as páginas Thymeleaf, que trazem apenas a estrutura das telas; os scripts da página chamam a API REST, protegida pelo Spring Security, e montam os indicadores e gráficos. Clientes externos, como o Postman, acessam a mesma API com HTTP Basic. Os controllers delegam aos serviços, que aplicam as regras de domínio e consultam o MongoDB Atlas por meio dos repositórios. A Figura 1 apresenta essa organização.

Figura 1 - Arquitetura em camadas do dashboard Thindesk



Fonte: Elaborado pelo autor (2026).

O código fica no pacote `br.com.danielcardoso.thindesk`, dividido conforme o Quadro 4.

Quadro 4 - Pacotes do projeto

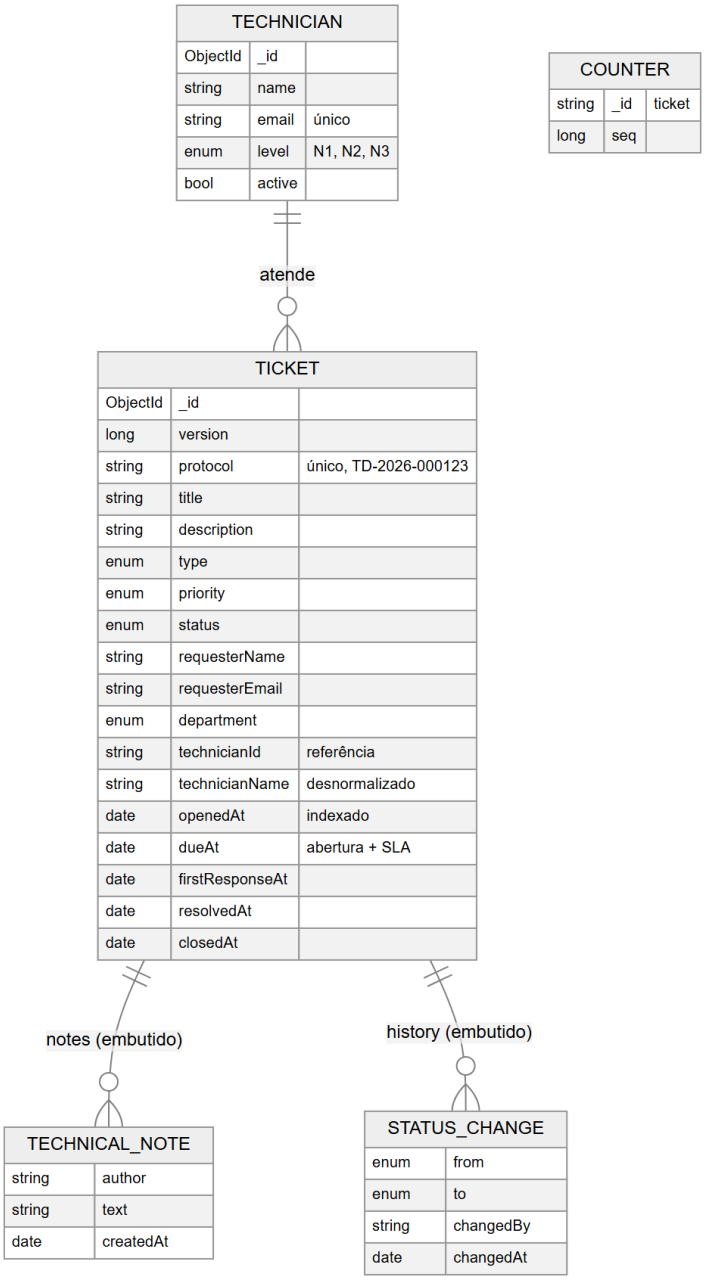
Pacote	Responsabilidade
ticket.domain	Entidades Ticket e Technician, enumerações de situação, prioridade, tipo e setor, e as regras do fluxo de situações e do SLA
ticket.repository	Repositórios Spring Data e consultas dinâmicas com Criteria (filtros, paginação e contador de protocolo)
ticket.service e ticket.dto	Casos de uso dos chamados, filtros normalizados e corpos de requisição e resposta
dashboard	DashboardService, que orquestra as consultas, e DashboardMetrics, que calcula indicadores, série histórica e rankings
api	Controllers REST e tratamento padronizado de erros
web	Controllers das páginas do painel
bootstrap	Criação do administrador inicial e da massa de demonstração
config, security, user, admin, theme	Base reaproveitada da atividade de login, adaptada para a API

Fonte: Elaborado pelo autor (2026).

4.2 Modelo de dados

O MongoDB armazena documentos JSON em coleções (MONGODB, 2026). O chamado é o documento central: as considerações técnicas e o histórico de situações ficam embutidos nele, porque só existem dentro do chamado e são sempre lidos junto com ele. O técnico é referenciado pelo identificador, e o nome também é gravado no chamado (desnormalização) para que listas e rankings não precisem consultar outra coleção. A coleção `counters` guarda o contador atômico usado no protocolo. A Figura 2 mostra o modelo.

Figura 2 - Modelo de dados das coleções do MongoDB



Fonte: Elaborado pelo autor (2026).

O Quadro 5 traz um exemplo resumido de documento da coleção `tickets`.

Quadro 5 - Exemplo de documento da coleção tickets

```
{
  "_id": "6af1d146982ced63ebadb715",
  "version": 4,
  "protocol": "TD-2026-001233",
  "title": "Teclado com teclas falhando",
  "type": "HARDWARE",
  "priority": "LOW",
  "status": "IN_PROGRESS",
  "requesterName": "João Pedro Santos",
  "department": "SALES",
  "technicianId": "6af1d145982ced63ebadb240",
  "technicianName": "Rafael Oliveira",
  "openedAt": "2026-10-19T11:20:00Z",
  "dueAt": "2026-10-22T11:20:00Z",
  "firstResponseAt": "2026-10-19T14:02:00Z",
  "notes": [ { "author": "Rafael Oliveira",
               "text": "Acesso remoto realizado. Verificando a estação.",
               "createdAt": "2026-10-19T14:02:00Z" } ],
  "history": [ { "from": "OPEN", "to": "IN_PROGRESS",
                 "changedBy": "Rafael Oliveira",
                 "changedAt": "2026-10-19T14:02:00Z" } ]
}
```

Fonte: Elaborado pelo autor (2026).

4.3 Regras de negócio

Cada prioridade tem um prazo de solução, o SLA, que define a data limite do chamado no momento da abertura, conforme o Quadro 6.

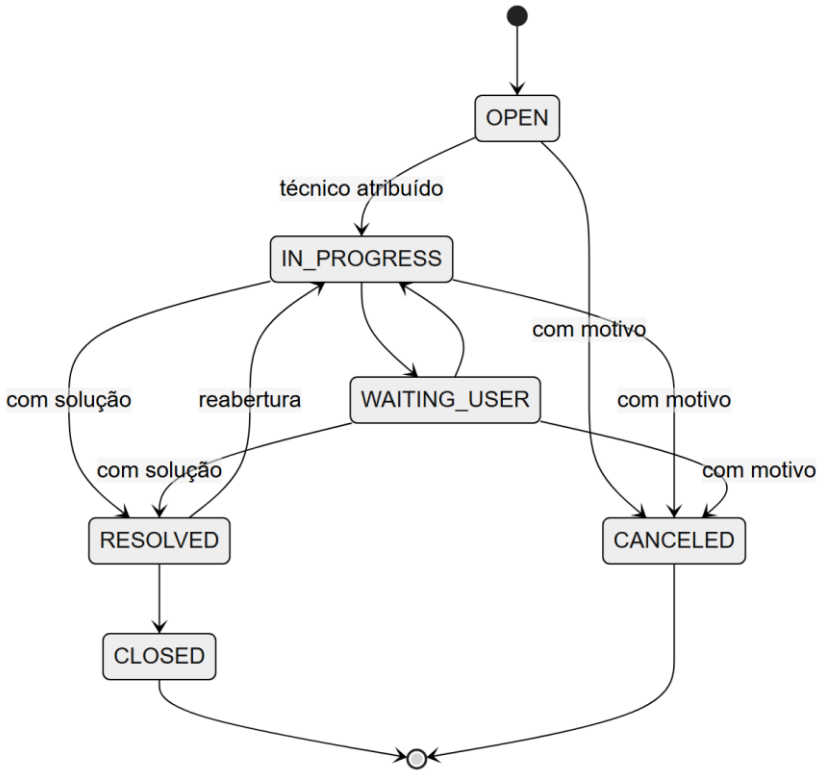
Quadro 6 - Prazo de solução (SLA) por prioridade

Prioridade	Código	Prazo de solução
Baixa	LOW	72 horas
Média	MEDIUM	24 horas
Alta	HIGH	8 horas
Crítica	CRITICAL	4 horas

Fonte: Elaborado pelo autor (2026).

A situação do chamado segue o fluxo da Figura 3. Iniciar o atendimento exige um técnico atribuído; resolver exige registrar a solução; cancelar exige informar o motivo. Chamados fechados ou cancelados não aceitam novas considerações, transferência nem mudança de situação, e técnicos inativos não recebem chamados. As datas usadas nos indicadores são gravadas pelo próprio domínio: primeira resposta ao iniciar o atendimento, solução ao resolver (apagada na reabertura) e encerramento ao fechar ou cancelar.

Figura 3 - Fluxo de situações do chamado



Fonte: Elaborado pelo autor (2026).

Para que todos os indicadores falem a mesma língua, os termos usados no painel foram definidos conforme o Quadro 7.

Quadro 7 - Definições usadas nos indicadores

Termo	Definição
Em aberto (backlog)	Situação Aberto, Em atendimento ou Aguardando usuário
Atendido	Situação Resolvido ou Fechado (tem solução registrada)
Prazo (SLA)	Data de abertura somada às horas de SLA da prioridade
Atrasado	Em aberto com o prazo vencido
SLA cumprido	Atendidos com solução até o prazo, sobre o total de atendidos
Taxa de resolução	Atendidos sobre o total de chamados não cancelados
Primeira resposta	Tempo entre a abertura e o início do atendimento
Tempo de solução	Tempo entre a abertura e a solução

Fonte: Elaborado pelo autor (2026).

4.4 Versionamento com Gitflow

O código foi versionado no GitHub seguindo o fluxo Gitflow (ATLASSIAN, 2026), com uma branch por funcionalidade integrada à branch de desenvolvimento e uma branch de release para a versão estável, conforme o Quadro 8. O repositório está disponível em https://github.com/danrerez/thindesk-dashboard.

Quadro 8 - Branches do repositório

Branch	Conteúdo
main	Versões estáveis, marcadas com tags (v1.0.0 e v1.0.1)
develop	Integração do desenvolvimento
feature/base-login	Base reaproveitada da atividade de login, renomeada para Thindesk
feature/ticket-domain	Domínio de chamados, regras de SLA e fluxo, repositórios e massa de demonstração
feature/dashboard-api	Endpoints REST, filtros, cálculos, erros padronizados, segurança e OpenAPI
feature/dashboard-ui	Telas do dashboard, lista e detalhe de chamados com Chart.js
feature/tests	Testes automatizados e roteiro de integração
feature/docs	README, documentação, evidências e coleção Postman
release/1.0.0	Preparação da versão 1.0.0, mesclada em main e develop
hotfix/1.0.1	Chaves de configuração no arquivo .env.example

Fonte: Elaborado pelo autor (2026).

5 API REST

A API segue o estilo arquitetural REST (FIELDING, 2000): os recursos são identificados por URLs, as operações usam os métodos HTTP e as respostas são representações em JSON. A URL base é `http://localhost:8080/api/v1`. A documentação interativa, gerada a partir do código no padrão OpenAPI 3, fica em `/swagger-ui.html` (SPRINGDOC, 2026), e uma coleção Postman acompanha o repositório.

5.1 Autenticação e autorização

Todos os endpoints exigem um usuário com perfil MANAGER, que representa o responsável pelos chamados, ou ADMIN. Há duas formas de autenticação, descritas no Quadro 9.

Quadro 9 - Formas de autenticação da API

Forma	Quando usar	Proteção CSRF
Sessão (cookie SESSION)	O próprio dashboard, após o login em /login	Obrigatória em POST e PATCH: cabeçalho X-CSRF-TOKEN com o token publicado na página
HTTP Basic	Postman, curl e integrações	Dispensada: o navegador não envia esse cabeçalho sozinho a outro site, então não há risco de CSRF (OWASP FOUNDATION, 2026)

Fonte: Elaborado pelo autor (2026).

Sem credenciais, a API responde 401 em JSON, sem redirecionar para a tela de login e sem o cabeçalho WWW-Authenticate, para que o navegador não abra a janela nativa de senha. Um perfil sem permissão recebe 403.

5.2 Convenções e filtros

Os filtros do Quadro 10 são aceitos por todos os endpoints do dashboard, exceto o de opções de filtro, e pela listagem de chamados. Todos são opcionais e combináveis: filtros diferentes se somam (E lógico) e valores de um mesmo filtro se alternam (OU lógico). Listas aceitam valores separados por vírgula ou o parâmetro repetido.

Quadro 10 - Filtros comuns (RF05)

Parâmetro	Tipo	Descrição
from	data yyyy-MM-dd	Data inicial de abertura (inclusiva)
to	data yyyy-MM-dd	Data final de abertura (inclusiva)
status	lista	OPEN, IN_PROGRESS, WAITING_USER, RESOLVED, CLOSED, CANCELED
type	lista	HARDWARE, SOFTWARE, NETWORK, ACCESS, EMAIL, PRINTER, ERP, PHONE
priority	lista	LOW, MEDIUM, HIGH, CRITICAL
department	lista	FINANCE, HR, SALES, OPERATIONS, MARKETING, LOGISTICS, LEGAL, BOARD
technicianId	texto	Identificador do técnico ou none para chamados sem técnico
q	texto (até 80)	Busca em protocolo, título, solicitante e técnico
overdue	booleano	Somente chamados em aberto com prazo vencido

Fonte: Elaborado pelo autor (2026).

As datas dos filtros são interpretadas no fuso America/Sao_Paulo; a data inicial deve ser anterior ou igual à final, e o período pode ter no máximo 731 dias. Nas respostas, instantes seguem a ISO 8601 em UTC, tempos são expressos em horas com uma casa decimal e percentuais vão de 0 a 100. Médias e percentuais vêm nulos quando não há dados para calculá-los, em vez de zero, que seria enganoso. Enumerações são devolvidas como objeto com código e rótulo em português, para que o front-end não precise traduzi-las.

5.3 Tratamento de erros

Todos os erros seguem o padrão Problem Details for HTTP APIs (NOTTINGHAM; WILDE; DALAL, 2023), com o tipo de conteúdo `application/problem+json`, sempre com os campos `code` e `timestamp`. Erros de validação trazem também a lista `errors` campo a campo, como no Quadro 11.

Quadro 11 - Exemplo de resposta de erro de validação

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "/docs/errors#validation-error",
  "title": "Parâmetros inválidos",
  "status": 400,
  "detail": "Valor inválido 'ABERTO'. Valores aceitos: OPEN, IN_PROGRESS, ...",
  "instance": "/api/v1/dashboard/summary",
  "code": "VALIDATION_ERROR",
  "timestamp": "2026-11-09T14:00:06.493Z",
  "errors": [ { "field": "status",
                "message": "Valor inválido 'ABERTO'. Valores aceitos: ..." } ]
}
```

Fonte: Elaborado pelo autor (2026).

O Quadro 12 relaciona os códigos de erro possíveis. A separação entre 400 e 422 distingue uma requisição mal formada de uma requisição válida que o estado atual do chamado não permite.

Quadro 12 - Códigos de erro da API

HTTP	code	Quando acontece
400	VALIDATION_ERROR	Parâmetro ou campo inválido: data fora do formato, enumeração inexistente, data inicial depois da final, limit ou size fora do intervalo, ordenação não permitida, campo obrigatório ausente
400	MALFORMED_BODY	JSON malformado ou com valor de enumeração inválido no corpo
401	UNAUTHORIZED	Sem sessão e sem credenciais, ou credenciais erradas
403	FORBIDDEN	Usuário sem perfil MANAGER ou ADMIN
403	CSRF_TOKEN_INVALID	POST ou PATCH com sessão sem o cabeçalho X-CSRF-TOKEN
404	NOT_FOUND	Chamado inexistente ou rota que não existe
405	METHOD_NOT_ALLOWED	Método HTTP não aceito no endpoint
409	CONCURRENT_UPDATE	Chamado alterado por outra pessoa entre a leitura e a gravação
415	UNSUPPORTED_MEDIA_TYPE	Corpo enviado sem application/json
422	BUSINESS_RULE	Requisição válida que viola uma regra de negócio, como transição não permitida ou resolução sem solução
500	INTERNAL_ERROR	Erro inesperado, com detalhes apenas no log do servidor

Fonte: Elaborado pelo autor (2026).

5.4 Visão geral dos endpoints

O Quadro 13 resume os 13 endpoints. Cada bloco do dashboard tem um endpoint próprio, o que permite carregar e tratar falhas de cada bloco de forma independente.

Quadro 13 - Endpoints da API (prefixo /api/v1)

Método	Endpoint	Descrição	RF
GET	/dashboard/summary	Indicadores gerais	RF01
GET	/dashboard/timeline	Abertos e atendidos por período	RF02
GET	/dashboard/status	Situação e backlog por prioridade	RF03
GET	/dashboard/types	Tipos de chamados mais atendidos	RF04
GET	/dashboard/technicians	Desempenho por técnico	RF01, RF04
GET	/dashboard/filters	Opções dos filtros	RF05
GET	/tickets	Lista paginada de chamados	RF03, RF05
GET	/tickets/{id}	Detalhe do chamado	RF03
POST	/tickets	Abre um chamado	RF07
PATCH	/tickets/{id}/status	Altera a situação	RF07
PATCH	/tickets/{id}/assignment	Atribui ou transfere o técnico	RF07
POST	/tickets/{id}/notes	Registra consideração técnica	RF07
GET	/technicians	Lista os técnicos	RF05

Fonte: Elaborado pelo autor (2026).

5.5 Indicadores gerais

`GET /dashboard/summary` (RF01) devolve os totais por situação, o backlog, os atrasados, a taxa de resolução, o SLA cumprido e os tempos médios. Quando `from` e `to` são informados, inclui a comparação com o período anterior de mesma duração e mesmos filtros; caso contrário, `comparison` vem nulo. Aceita os filtros comuns. Erros possíveis: 400, 401 e 403. O Quadro 14 mostra uma requisição e a resposta obtida no banco de demonstração.

Quadro 14 - Requisição e resposta de GET /dashboard/summary

```
GET /api/v1/dashboard/summary?from=2026-10-11&to=2026-11-09

200 OK
{
  "period": { "from": "2026-10-11", "to": "2026-11-09" },
  "totals": { "total": 226, "active": 16, "open": 3, "inProgress": 11,
              "waitingUser": 2, "solved": 204, "canceled": 6, "overdue": 12,
              "criticalActive": 1, "unassignedActive": 3 },
  "rates": { "resolutionRate": 92.7, "slaMetPercent": 66.7 },
  "times": { "avgFirstResponseHours": 7.6, "avgResolutionHours": 32.7,
             "medianResolutionHours": 16.3 },
  "comparison": { "previousPeriod": { "from": "2026-09-11", "to": "2026-10-10" },
                  "previousTotal": 259, "previousSolved": 247,
                  "totalChangePercent": -12.7, "solvedChangePercent": -17.4 },
  "generatedAt": "2026-11-09T14:09:27.847Z"
}
```

Fonte: Elaborado pelo autor (2026).

5.6 Série histórica

`GET /dashboard/timeline` (RF02) conta os chamados abertos pela data de abertura e os atendidos pela data de solução, cada um dentro do período, e preenche com zero os intervalos sem movimento. Além dos filtros comuns, aceita o parâmetro do Quadro 15. Erros possíveis: 400 para granularidade inexistente ou período que geraria mais de 400 pontos, 401 e 403.

Quadro 15 - Parâmetro específico de GET /dashboard/timeline

Parâmetro	Valores	Padrão	Descrição
granularity	DAY, WEEK, MONTH	automático	Até 45 dias agrupa por dia, até 200 por semana e acima disso por mês. Semanas começam na segunda-feira

Fonte: Elaborado pelo autor (2026).

Quadro 16 - Requisição e resposta resumida de GET /dashboard/timeline

```
GET /api/v1/dashboard/timeline?from=2026-10-11&to=2026-11-09&granularity=WEEK

200 OK
{
  "granularity": { "code": "WEEK", "label": "Semana" },
  "period": { "from": "2026-10-11", "to": "2026-11-09" },
  "points": [
    { "period": "2026-10-12", "label": "Sem. 12/10", "opened": 57, "solved": 51,
      "avgResolutionHours": 32.2 },
    { "period": "2026-10-19", "label": "Sem. 19/10", "opened": 51, "solved": 52,
      "avgResolutionHours": 34.6 }
  ],
  "totalOpened": 226,
  "totalSolved": 212
}
```

Fonte: Elaborado pelo autor (2026).

5.7 Situação dos chamados

`GET /dashboard/status` (RF03) devolve a quantidade e o percentual de cada situação, inclusive as zeradas, e o backlog por prioridade com quantos chamados já estão atrasados. Aceita os filtros comuns. Erros possíveis: 400, 401 e 403.

Quadro 17 - Resposta resumida de GET /dashboard/status

```
200 OK
{
  "total": 226,
  "items": [
    { "code": "OPEN", "label": "Aberto", "count": 3, "percent": 1.3, "active": true },
    { "code": "IN_PROGRESS", "label": "Em atendimento", "count": 11, "percent": 4.9,
      "active": true },
    { "code": "CLOSED", "label": "Fechado", "count": 196, "percent": 86.7,
      "active": false }
  ],
  "activeByPriority": [
    { "code": "CRITICAL", "label": "Crítica", "slaHours": 4, "active": 1, "overdue": 1 },
    { "code": "LOW", "label": "Baixa", "slaHours": 72, "active": 7, "overdue": 7 }
  ]
}
```

Fonte: Elaborado pelo autor (2026).

5.8 Tipos mais atendidos e desempenho por técnico

`GET /dashboard/types` (RF04) ordena os tipos de chamado por atendidos ou por abertos e devolve total, atendidos, em aberto, participação, tempo médio de solução e SLA cumprido. `GET /dashboard/technicians` devolve, por técnico, os chamados atribuídos, atendidos, em aberto e atrasados, com tempo médio e SLA. Ambos aceitam os filtros comuns e os parâmetros do Quadro 18. Erros possíveis: 400, 401 e 403.

Quadro 18 - Parâmetros de ranking

Endpoint	Parâmetro	Valores	Padrão
/dashboard/types	metric	SOLVED ou TOTAL	SOLVED
/dashboard/types	limit	1 a 20	8
/dashboard/technicians	limit	1 a 50	10

Fonte: Elaborado pelo autor (2026).

Quadro 19 - Resposta resumida de GET /dashboard/types?limit=2

```
200 OK
{
  "metric": { "code": "SOLVED", "label": "Chamados atendidos (resolvidos ou fechados)" },
  "total": 204,
  "items": [
    { "position": 1, "code": "SOFTWARE", "label": "Software e instalação", "total": 43,
      "solved": 39, "active": 3, "share": 19.1, "avgResolutionHours": 50.2,
      "slaMetPercent": 53.8 },
    { "position": 2, "code": "HARDWARE", "label": "Hardware", "total": 36,
      "solved": 33, "active": 3, "share": 16.2, "avgResolutionHours": 30.8,
      "slaMetPercent": 36.4 }
  ]
}
```

Fonte: Elaborado pelo autor (2026).

5.9 Opções dos filtros

`GET /dashboard/filters` (RF05) devolve tudo o que o front-end precisa para montar a barra de filtros: situações, tipos, prioridades com SLA, setores, técnicos, granularidades, o intervalo de datas com dados, o fuso horário e o intervalo de atualização automática. Erros possíveis: 401 e 403.

Quadro 20 - Resposta resumida de GET /dashboard/filters

```
200 OK
{
  "statuses": [ { "code": "OPEN", "label": "Aberto" }, ... ],
  "types": [ { "code": "HARDWARE", "label": "Hardware" }, ... ],
  "priorities": [ { "code": "LOW", "label": "Baixa", "slaHours": 72 }, ... ],
  "departments": [ { "code": "FINANCE", "label": "Financeiro" }, ... ],
  "technicians": [ { "id": "6af1d145982ced63ebadb23c", "name": "Ana Beatriz Souza",
                     "level": "N1", "active": true }, ... ],
  "dataRange": { "firstOpenedAt": "2026-05-14", "lastOpenedAt": "2026-11-09",
                 "totalTickets": 1401 },
  "timezone": "America/Sao_Paulo",
  "refreshSeconds": 60
}
```

Fonte: Elaborado pelo autor (2026).

5.10 Consulta de chamados

`GET /tickets` lista os chamados de forma paginada, sem descrição, notas e histórico. Além dos filtros comuns, aceita os parâmetros do Quadro 21. `GET /tickets/{id}` devolve o detalhe completo, com as considerações técnicas, o histórico e as próximas situações permitidas (`allowedTransitions`), usadas pela tela para

montar as ações. Erros possíveis: 400 (ordenação, tamanho de página ou filtros inválidos), 401, 403 e 404 para chamado inexistente.

Quadro 21 - Parâmetros de paginação de GET /tickets

Parâmetro	Valores	Padrão	Descrição
page	a partir de 0	0	Página
size	1 a 100	20	Itens por página
sort	campo,direção	openedAt,desc	Campos: openedAt, dueAt, resolvedAt, protocol, status, type; direção asc ou desc

Fonte: Elaborado pelo autor (2026).

Quadro 22 - Resposta resumida de GET /tickets/{id}

```
200 OK
{
  "id": "6af1d1fe99272fca98b3f0c0",
  "protocol": "TD-2026-001401",
  "status": { "code": "RESOLVED", "label": "Resolvido" },
  "priority": { "code": "HIGH", "label": "Alta" },
  "technician": { "id": "6af1d145982ced63ebadb23c", "name": "Ana Beatriz Souza" },
  "openedAt": "2026-11-09T14:00:30Z",
  "dueAt": "2026-11-09T22:00:30Z",
  "resolvedAt": "2026-11-09T14:06:20Z",
  "overdue": false, "slaMet": true, "slaHours": 8,
  "allowedTransitions": [ { "code": "IN_PROGRESS", "label": "Em atendimento" },
                          { "code": "CLOSED", "label": "Fechado" } ],
  "notes": [ { "author": "Admin Demo",
               "text": "Porta do switch reconfigurada na VLAN correta.",
               "createdAt": "2026-11-09T14:06:20Z" } ],
  "history": [ { "from": { "code": "IN_PROGRESS", "label": "Em atendimento" },
                 "to": { "code": "RESOLVED", "label": "Resolvido" },
                 "changedBy": "Admin Demo", "changedAt": "2026-11-09T14:06:20Z" } ]
}
```

Fonte: Elaborado pelo autor (2026).

5.11 Atualização de chamados

`POST /tickets` abre um chamado (RF07). O protocolo é gerado em sequência no formato TD-ano-número, o prazo é calculado pela prioridade, nome e e-mail são normalizados e uma consideração técnica de abertura é registrada. A resposta é 201 com o cabeçalho Location e o detalhe do chamado. Os campos aceitos estão no Quadro 23.

Quadro 23 - Campos de POST /tickets

Campo	Obrigatório	Regra
title	sim	5 a 120 caracteres
description	sim	até 4000 caracteres
type	sim	enumeração de tipo
priority	sim	enumeração de prioridade
requesterName	sim	até 80 caracteres
requesterEmail	sim	e-mail válido, gravado em minúsculas
department	sim	enumeração de setor
technicianId	não	técnico ativo existente

Fonte: Elaborado pelo autor (2026).

Quadro 24 - Requisição de POST /tickets

```
POST /api/v1/tickets
Content-Type: application/json

{
  "title": "Sem acesso à pasta de rede",
  "description": "A unidade compartilhada do financeiro não abre desde o almoço.",
  "type": "NETWORK",
  "priority": "HIGH",
  "requesterName": "Mariana Costa",
  "requesterEmail": "Mariana.Costa@Exemplo.com.br",
  "department": "FINANCE"
}
```

Fonte: Elaborado pelo autor (2026).

`PATCH /tickets/{id}/status` altera a situação com o corpo `{ "status": ..., "note": ... }`, em que a nota é obrigatória para resolver ou cancelar. `PATCH /tickets/{id}/assignment` recebe `{ "technicianId": ... }` e registra uma nota automática de atribuição ou transferência. `POST /tickets/{id}/notes` recebe `{ "text": ... }` e responde 201. Todos devolvem 400 para corpo inválido, 404 para chamado inexistente, 409 para atualização concorrente, 401 e 403, além dos erros de regra de negócio do Quadro 25.

Quadro 25 - Erros de regra de negócio (HTTP 422)

Situação	Mensagem (detail)
Iniciar o atendimento sem técnico	Atribua um técnico antes de iniciar o atendimento.
Transição fora do fluxo	Não é possível passar o chamado de Aberto para Resolvido.
Mesma situação	O chamado já está com a situação Em atendimento.
Resolver sem solução	Registre a solução aplicada (campo note) para resolver o chamado.
Cancelar sem motivo	Informe o motivo do cancelamento (campo note).
Técnico inexistente ou inativo	Técnico ... não encontrado. / O técnico ... está inativo.
Nota ou atribuição em chamado finalizado	Chamados fechados ou cancelados não recebem novas considerações.

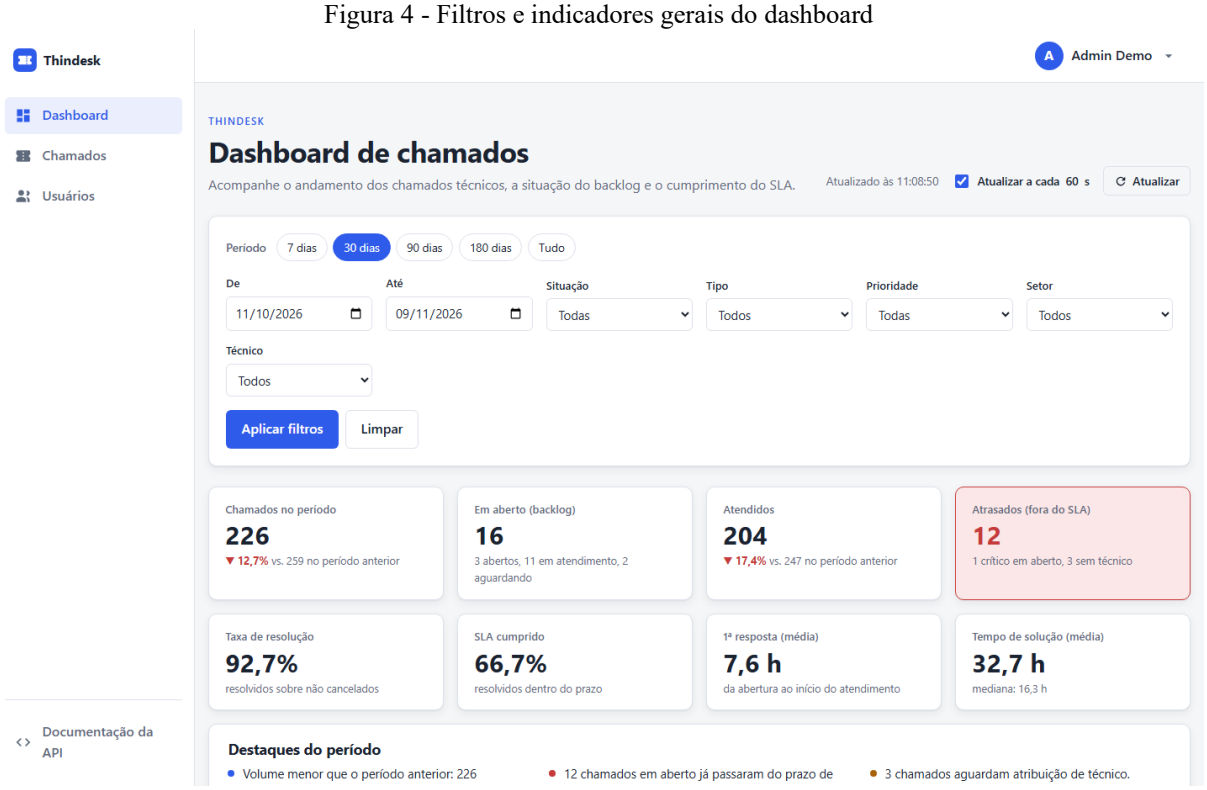
Fonte: Elaborado pelo autor (2026).

6 DASHBOARD ADMINISTRATIVO

As telas foram construídas em Thymeleaf (THYMELEAF, 2026) com o mesmo layout e os temas visuais da atividade de login. Elas trazem apenas a estrutura; todos os dados são buscados na API pelos scripts da página, e os gráficos são desenhados com a biblioteca Chart.js (CHART.JS, 2026). Os dados exibidos são a massa de demonstração gerada automaticamente: 9 técnicos e 1.400 chamados distribuídos nos 180 dias anteriores à execução.

A Figura 4 mostra o topo do dashboard, com a barra de filtros (RF05) e os indicadores gerais (RF01). O card de atrasados fica destacado quando há chamados fora do prazo, e cada indicador traz a comparação com o período anterior ou um detalhamento.

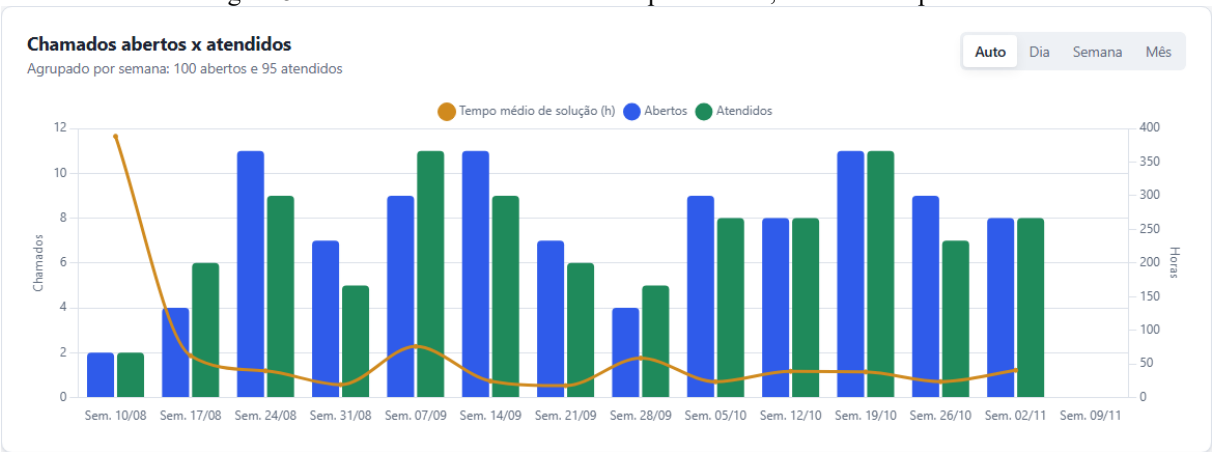
Figura 4 - Filtros e indicadores gerais do dashboard



Fonte: Elaborado pelo autor (2026).

Logo abaixo, o quadro de destaques transforma os números em frases que apoiam a decisão, como o volume em relação ao período anterior, o tipo com pior SLA e o técnico com a maior fila. Em seguida vem o gráfico de chamados abertos e atendidos por período (RF02). A Figura 5 mostra o gráfico com filtros aplicados: 90 dias, apenas chamados de hardware, agrupados por semana automaticamente.

Figura 5 - Chamados abertos e atendidos por semana, com filtros aplicados



Fonte: Elaborado pelo autor (2026).

A Figura 6 mostra a situação dos chamados e o backlog por prioridade (RF03). Cada item da legenda abre a lista de chamados já filtrada por aquela situação.

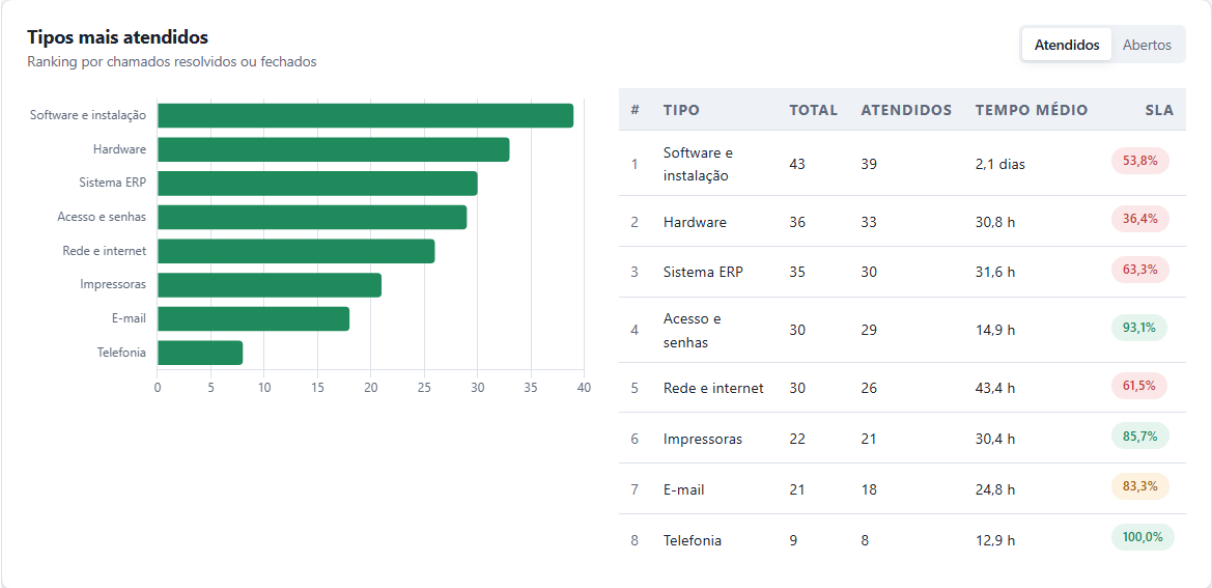
Figura 6 - Situação dos chamados e backlog por prioridade



Fonte: Elaborado pelo autor (2026).

A Figura 7 apresenta o ranking dos tipos mais atendidos (RF04), com o SLA de cada tipo destacado por cor, e a Figura 8 mostra o desempenho por técnico.

Figura 7 - Ranking dos tipos de chamados mais atendidos



#	TIPO	TOTAL	ATENDIDOS	TEMPO MÉDIO	SLA
1	Software e instalação	43	39	2,1 dias	53,8%
2	Hardware	36	33	30,8 h	36,4%
3	Sistema ERP	35	30	31,6 h	63,3%
4	Acesso e senhas	30	29	14,9 h	93,1%
5	Rede e internet	30	26	43,4 h	61,5%
6	Impressoras	22	21	30,4 h	85,7%
7	E-mail	21	18	24,8 h	83,3%
8	Telefonia	9	8	12,9 h	100,0%

Fonte: Elaborado pelo autor (2026).

Figura 8 - Desempenho por técnico

Desempenho por técnico

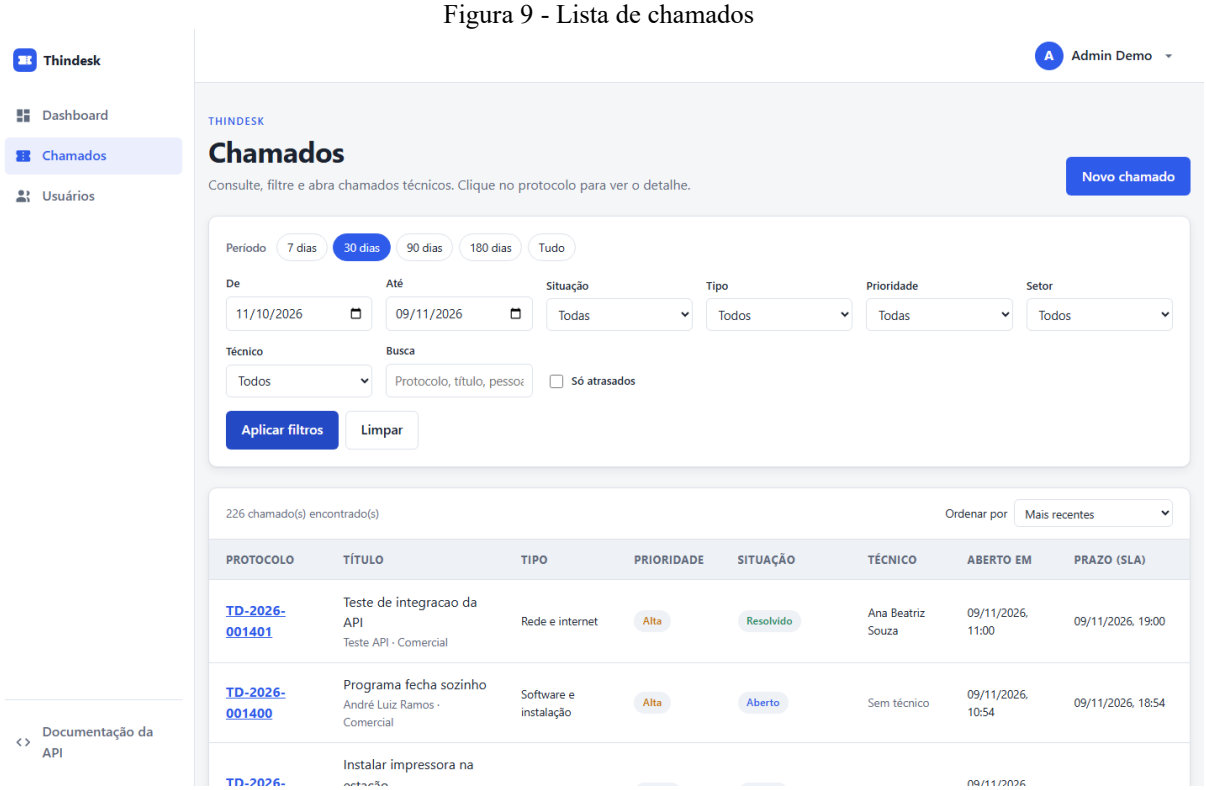
Ordenado por chamados atendidos. 3 chamados em aberto sem técnico.

#	TÉCNICO	ATRIBUÍDOS	ATENDIDOS	EM ABERTO	TEMPO MÉDIO	SLA
1	Juliana Martins	56	50	5 (5 atrasados)	2,0 dias	68,0%
2	Rafael Oliveira	37	34	1 (1 atrasado)	35,4 h	64,7%
3	Thiago Almeida	30	28	1	33,8 h	57,1%
4	Ana Beatriz Souza	31	27	3 (3 atrasados)	28,4 h	74,1%
5	Lucas Ferreira	28	26	1 (1 atrasado)	36,5 h	61,5%
6	Camila Rocha	24	22	2 (2 atrasados)	10,1 h	77,3%
7	Patrícia Gomes	11	11	0	5,5 h	63,6%
8	Marcos Vinícius Lima	6	6	0	13,1 h	66,7%

Fonte: Elaborado pelo autor (2026).

A tela de chamados (Figura 9) lista os registros com filtros, busca textual, ordenação, paginação e o botão para abrir um novo chamado. Os chamados atrasados ficam marcados na lista.

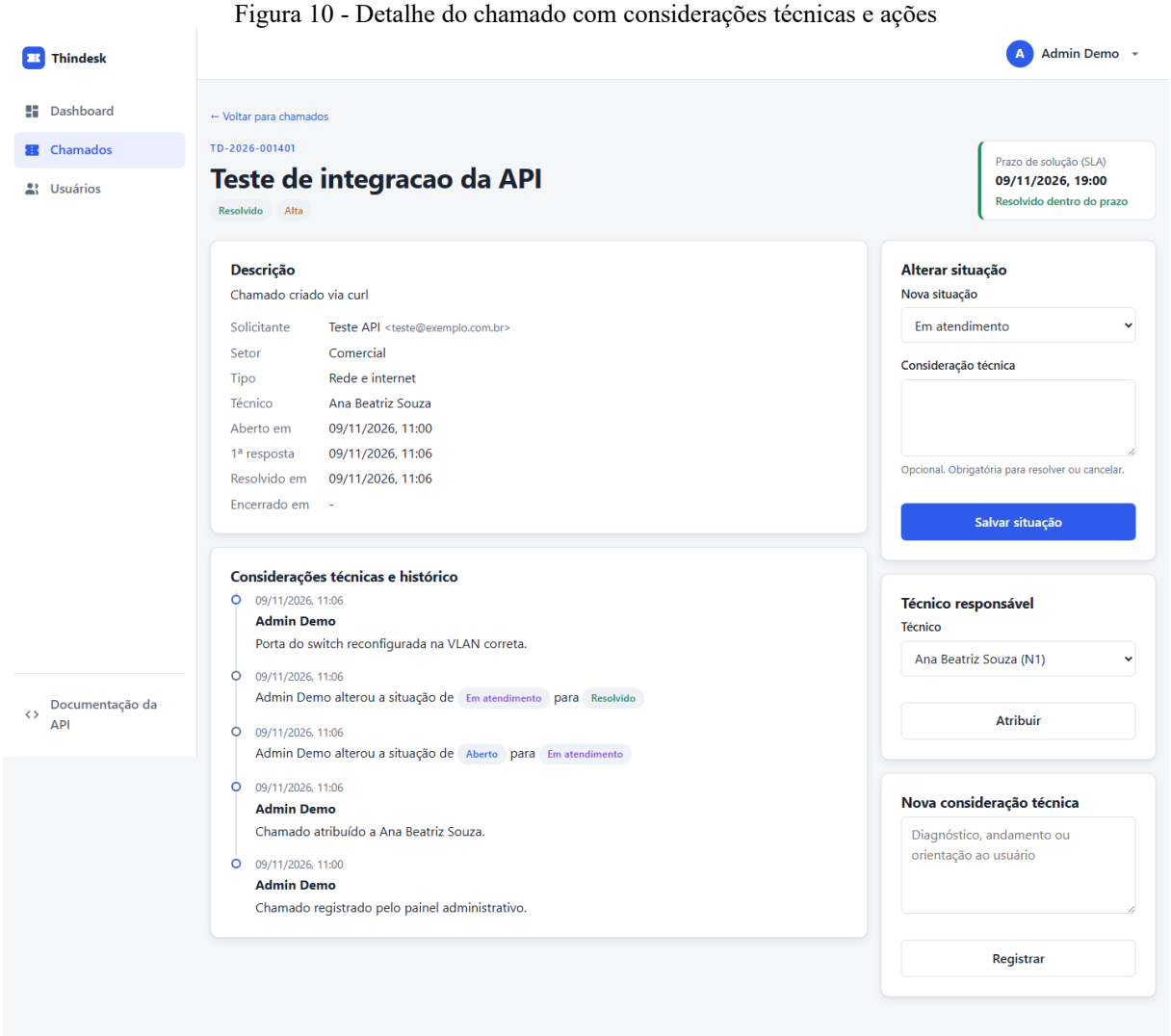
Figura 9 - Lista de chamados



Fonte: Elaborado pelo autor (2026).

O detalhe do chamado (Figura 10) mostra a descrição, os dados do solicitante, as datas, a situação do SLA e a linha do tempo com as considerações técnicas e o histórico de situações. Na lateral ficam as ações de alterar situação, atribuir técnico e registrar uma nova consideração técnica (RF07); as opções de situação vêm da própria API, e as mensagens de regra de negócio são exibidas no formulário.

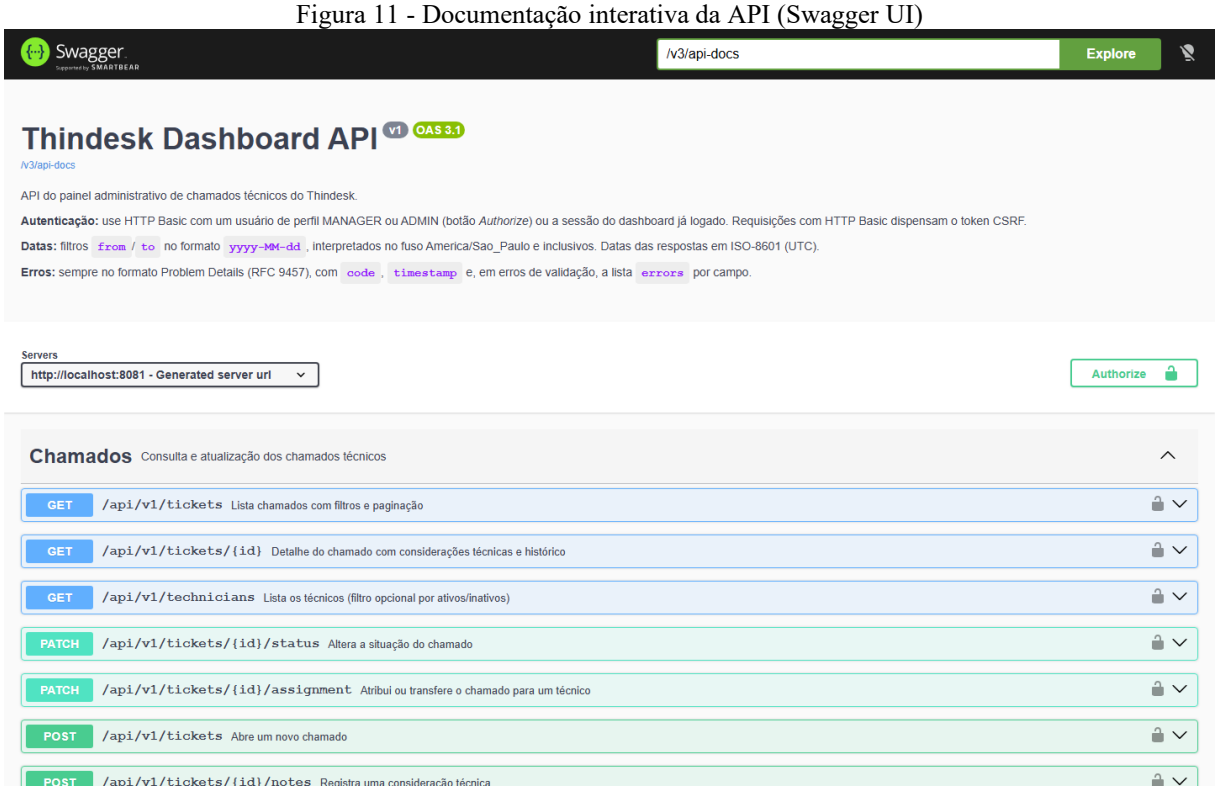
Figura 10 - Detalhe do chamado com considerações técnicas e ações



Fonte: Elaborado pelo autor (2026).

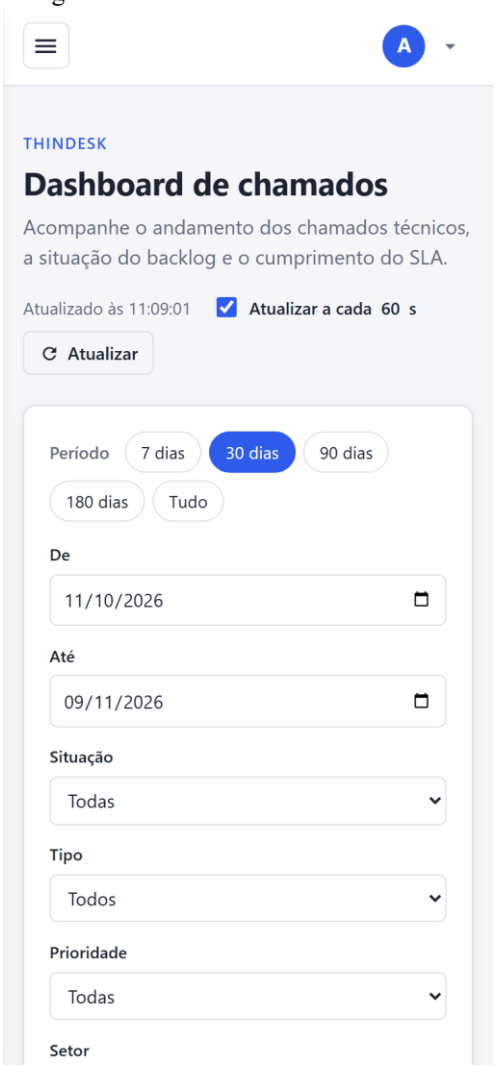
A documentação interativa da API (Figura 11) permite testar os endpoints no próprio navegador, autenticando com HTTP Basic. Por fim, a Figura 12 mostra o dashboard em um celular, sem rolagem horizontal.

Figura 11 - Documentação interativa da API (Swagger UI)



Fonte: Elaborado pelo autor (2026).

Figura 12 - Dashboard em tela de celular



Fonte: Elaborado pelo autor (2026).

7 PRINCIPAIS DECISÕES DE PROJETO

O Quadro 26 reúne as decisões mais relevantes tomadas durante a análise e o projeto, com as respectivas justificativas.

Quadro 26 - Decisões de projeto e justificativas

Decisão	Justificativa
Reaproveitar a base da atividade de login	Ela foi construída para ser reutilizada: o painel herdou autenticação, perfis, sessões no Atlas, layout e temas sem reescrevê-los. O responsável pelos chamados é o perfil MANAGER.
Notas e histórico embutidos no chamado	Só existem dentro do chamado e são sempre lidos com ele: um documento por chamado evita junções e mantém a gravação atômica.
Nome do técnico desnormalizado	Rankings e listas não precisam consultar a coleção de técnicos; o identificador continua sendo a referência.
Filtros no banco e cálculos em Java	O MongoDB aplica os filtros com índices e devolve só os campos necessários; os cálculos ficam em funções puras, testadas com cenários calculados à mão. Para volumes muito maiores, migrariam para o aggregation pipeline sem mudar a API.
Um endpoint por bloco do dashboard	Cada bloco carrega e falha de forma independente, os endpoints são reutilizáveis e a documentação corresponde aos requisitos.
Mesmos filtros em todos os endpoints	Um único objeto de filtros garante que todos os números do painel se refiram ao mesmo recorte.
Atendidos contados pela data de solução	Responde corretamente quanto foi atendido no período, inclusive chamados abertos antes dele.
Médias nulas sem dados	Evita exibir zero quando não há base de cálculo.
Páginas Thymeleaf com dados via fetch	Mantém a pilha da atividade anterior e torna explícita a integração com a API (RF06).
Sessão e HTTP Basic na API	O painel usa a sessão com CSRF; integrações usam Basic. Erros 401 e 403 em JSON, sem janela nativa do navegador.
Problem Details (RFC 9457)	Padrão oficial do HTTP, suportado pelo Spring, com um código estável para o front-end tratar cada caso.
400 para validação e 422 para regra de negócio	Separa requisição mal formada de requisição válida que o estado do chamado não permite.
Atualização automática a cada 60 segundos	Simples e suficiente para um painel gerencial; pausa com a aba oculta para não gerar carga desnecessária.
Relógio injetável e fuso configurável	Permite testar SLA e séries com data fixa e agrupar por dia no fuso de São Paulo, não em UTC.
Massa de demonstração determinística	Gera dados realistas pelas próprias regras de domínio, com semente fixa, para o painel ter conteúdo desde a primeira execução.

Fonte: Elaborado pelo autor (2026).

8 TESTES E VALIDAÇÃO

A solução foi validada em três frentes: testes automatizados, testes de integração contra o banco real e testes manuais pela interface.

8.1 Testes automatizados

Os testes automatizados usam JUnit 5, AssertJ, Mockito e MockMvc e não dependem do MongoDB Atlas: as regras de domínio e os cálculos são testados com dados montados à mão, e a camada web usa o Spring Security real com os serviços simulados. A execução de `./mvnw test` resultou em 75 testes aprovados, nenhuma falha e nenhum erro, distribuídos conforme o Quadro 27.

Quadro 27 - Testes automatizados

Classe	Testes	O que cobre
DashboardMetricsTest	13	Cenário de 7 chamados com resultados calculados à mão: totais, taxa de resolução, SLA, médias e mediana, séries diária e semanal, fuso de São Paulo, granularidade automática, limite de pontos, situação, rankings e ausência de dados
TicketWorkflowTest	8	Prazo de SLA, fluxo de situações, técnico obrigatório, datas de resposta, solução e fechamento, reabertura, atraso, técnico inativo e chamados finalizados
TicketServiceTest	5	Protocolo sequencial e normalização, solução obrigatória, nota de transferência e erros de técnico e chamado inexistentes
DashboardApiTest	18	API com o Spring Security real: 401 em JSON, 403 por perfil, HTTP Basic, validação de período, enumerações, datas, limit, sort e size, 404, 422, CSRF, 201 com Location, JSON malformado, 404 e 405 de rota, página do dashboard
Base de login (4 classes)	31	Perfis, telas iniciais, CSRF, cabeçalhos de segurança, BCrypt, bloqueio por tentativas e política de senha
Total	75	75 aprovados, 0 falhas, 0 erros

Fonte: Elaborado pelo autor (2026).

8.2 Testes de integração

O script `scripts/testes_api.py` executa 40 casos contra a aplicação conectada ao MongoDB Atlas: consulta todos os endpoints, verifica os erros documentados e leva um chamado de teste da abertura ao fechamento. Todos os 40 casos foram aprovados, conforme o Quadro 28.

Quadro 28 - Testes de integração da API contra o banco real

#	RF	Caso	Requisição	Esperado	Obtido	Resultado
1	RNF	Sem credenciais retorna 401 em JSON	GET /dashboard/summary	401	401	OK
2	RF01	Indicadores gerais do período	GET /dashboard/summary	200	200	OK

#	RF	Caso	Requisição	Esperado	Obtido	Resultado
3	RF01	Comparação com o período anterior	GET /dashboard/summary	200	200	OK
4	RF01	Indicadores sem período (todo o histórico)	GET /dashboard/summary	200	200	OK
5	RF02	Série diária (granularidade automática)	GET /dashboard/timeline	200	200	OK
6	RF02	Série semanal	GET /dashboard/timeline	200	200	OK
7	RF02	Série de 180 dias (automática: semanal)	GET /dashboard/timeline	200	200	OK
8	RF02	Granularidade inexistente	GET /dashboard/timeline	400	400	OK
9	RF02	DAY em período longo demais	GET /dashboard/timeline	400	400	OK
10	RF03	Situação dos chamados	GET /dashboard/status	200	200	OK
11	RF04	Tipos mais atendidos (top 3)	GET /dashboard/types	200	200	OK
12	RF04	Ranking por total aberto	GET /dashboard/types	200	200	OK
13	RF04	limit fora do intervalo	GET /dashboard/types	400	400	OK
14	RF04	Desempenho por técnico	GET /dashboard/technicians	200	200	OK
15	RF05	Opções dos filtros	GET /dashboard/filters	200	200	OK
16	RF05	Filtro combinado (situação, tipo, prioridade)	GET /dashboard/summary	200	200	OK
17	RF05	Busca textual na lista	GET /tickets	200	200	OK
18	RF05	Só atrasados, ordenado por prazo	GET /tickets	200	200	OK
19	RF05	Data inicial depois da final	GET /dashboard/summary	400	400	OK
20	RF05	Data em formato inválido	GET /dashboard/summary	400	400	OK
21	RF05	Situação inexistente	GET /dashboard/summary	400	400	OK
22	RF05	Ordenação por campo não permitido	GET /tickets	400	400	OK
23	RF05	Página grande demais	GET /tickets	400	400	OK
24	RF07	Abrir chamado	POST /tickets	201	201	OK
25	RF07	Corpo inválido lista erros por campo	POST /tickets	400	400	OK
26	RF07	Iniciar atendimento sem técnico	PATCH /tickets/{id}/status	422	422	OK
27	RF07	Transição fora do fluxo (Aberto > Resolvido)	PATCH /tickets/{id}/status	422	422	OK
28	RF07	Atribuir técnico	PATCH /tickets/{id}/assignment	200	200	OK
29	RF07	Técnico inexistente	PATCH /tickets/{id}/assignment	422	422	OK
30	RF07	Iniciar atendimento	PATCH /tickets/{id}/status	200	200	OK
31	RF07	Registrar consideração técnica	POST /tickets/{id}/notes	201	201	OK
32	RF07	Resolver sem informar a solução	PATCH /tickets/{id}/status	422	422	OK
33	RF07	Resolver com a solução	PATCH /tickets/{id}/status	200	200	OK
34	RF07	Fechar chamado	PATCH /tickets/{id}/status	200	200	OK
35	RF07	Nota em chamado fechado	POST /tickets/{id}/notes	422	422	OK
36	RF06	Detalhe com notas e histórico	GET /tickets/{id}	200	200	OK
37	RF06	Chamado inexistente	GET /tickets/{id}	404	404	OK
38	RF06	Rota inexistente	GET /nao-existe	404	404	OK
39	RF06	Método não suportado	DELETE /tickets/{id}	405	405	OK
40	RF06	JSON malformado	POST /tickets	400	400	OK

Fonte: Elaborado pelo autor (2026).

8.3 Testes manuais

Por fim, as funcionalidades foram verificadas na interface, com o resultado do Quadro 29.

Quadro 29 - Testes manuais pela interface

#	Verificação	Resultado
1	Login com perfil ADMIN abre o dashboard; visitante em /dashboard vai para o login	OK
2	Todos os blocos carregam sem erros no console do navegador	OK
3	Atalhos de período, filtros combinados e Limpar atualizam os blocos e a URL	OK
4	Data inicial maior que a final mostra erro no formulário e não consulta a API	OK
5	Botões Dia, Semana e Mês e Atendidos e Abertos mudam o gráfico e o ranking	OK
6	A atualização automática altera o horário da última atualização	OK
7	Legenda da situação e nome do técnico abrem a lista já filtrada	OK
8	Lista: busca, somente atrasados, ordenação e paginação	OK
9	Novo chamado com campos vazios mostra a mensagem de cada campo vinda da API	OK
10	Detalhe: atribuir, iniciar, tentar resolver sem solução e resolver com solução	OK
11	Layout em 390 px de largura sem rolagem horizontal	OK
12	Swagger UI executa chamadas autenticadas com HTTP Basic	OK

Fonte: Elaborado pelo autor (2026).

9 EXECUÇÃO DO PROJETO

Para executar o projeto são necessários o JDK 21 e uma conta no MongoDB Atlas; o Maven não precisa ser instalado, pois o projeto usa o Maven Wrapper. Os passos são:

a) clonar o repositório https://github.com/danrerez/thindesk-dashboard;

b) copiar o arquivo `.env.example` para `.env` e substituir cada chave `${YOUR_...}` pelo valor correspondente;

c) executar `./mvnw spring-boot:run` (ou `mvnw.cmd spring-boot:run` no Windows);

d) acessar http://localhost:8080 e entrar com o e-mail e a senha definidos para o administrador.

Na primeira execução são criados o administrador e a massa de demonstração. As variáveis de configuração estão no Quadro 30.

Quadro 30 - Variáveis de configuração (.env)

Variável	Descrição
MONGODB_URI	Connection string do MongoDB Atlas (obrigatória)
MONGODB_DATABASE	Nome do banco (padrão thindesk)
ADMIN_NAME, ADMIN_EMAIL, ADMIN_PASSWORD	Administrador criado no primeiro start
APP_TIMEZONE	Fuso dos filtros e gráficos (padrão America/Sao_Paulo)
DASHBOARD_REFRESH_SECONDS	Intervalo da atualização automática (padrão 60)
SEED_ENABLED, SEED_TICKETS, SEED_DAYS	Massa de demonstração (padrão 1.400 chamados em 180 dias)
SEED_RESET	true apaga chamados e técnicos e gera a massa de novo
APP_THEME	Tema visual: default ou dark

Fonte: Elaborado pelo autor (2026).

10 CONCLUSÃO

O dashboard administrativo do Thindesk atende aos sete requisitos funcionais propostos. O responsável pelos chamados acompanha os indicadores gerais, a evolução dos chamados abertos e atendidos ao longo do tempo, a situação atual do backlog, os tipos mais atendidos e o desempenho da equipe, com filtros que valem para todo o painel e dados atualizados automaticamente. A API REST projetada neste trabalho alimenta todas as telas e também pode ser usada por outros clientes, com contrato documentado, validação dos dados e erros padronizados.

Além dos requisitos mínimos, foram incorporados o quadro de destaques, a comparação com o período anterior, o desempenho por técnico, a tela de detalhe com as ações sobre o chamado e a documentação interativa da API. A solução foi comprovada por 75 testes automatizados, 40 casos de integração contra o banco real e testes manuais da interface, todos aprovados.

Como limitações e próximos passos, os cálculos são feitos na aplicação e, para volumes muito grandes, podem migrar para o aggregation pipeline do MongoDB sem alterar os endpoints; a atualização por consulta periódica pode evoluir para eventos enviados pelo servidor; e o bloqueio por tentativas de senha ainda não conta as tentativas feitas pelo HTTP Basic da API, embora contas bloqueadas já sejam recusadas. Também são evoluções naturais a exportação dos dados filtrados e metas de SLA configuráveis por tipo de chamado.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

ATLASSIAN. **Gitflow workflow**. 2026. Disponível em: https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow. Acesso em: 12 nov. 2026.

CHART.JS. **Chart.js documentation**. 2026. Disponível em: https://www.chartjs.org/docs/latest/. Acesso em: 12 nov. 2026.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Information and Computer Science) - University of California, Irvine, 2000.

MONGODB. **MongoDB Atlas documentation**. 2026. Disponível em: https://www.mongodb.com/docs/atlas/. Acesso em: 12 nov. 2026.

NOTTINGHAM, Mark; WILDE, Erik; DALAL, Sanjay. **RFC 9457**: problem details for HTTP APIs. [S. l.]: Internet Engineering Task Force, 2023. Disponível em: https://www.rfc-editor.org/rfc/rfc9457. Acesso em: 12 nov. 2026.

OWASP FOUNDATION. **Cross-site request forgery prevention cheat sheet**. 2026. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html. Acesso em: 12 nov. 2026.

SPRING. **Spring Boot reference documentation**. 2026a. Disponível em: https://docs.spring.io/spring-boot/index.html. Acesso em: 12 nov. 2026.

SPRING. **Spring Security reference**. 2026b. Disponível em: https://docs.spring.io/spring-security/reference/index.html. Acesso em: 12 nov. 2026.

SPRING. **Spring Data MongoDB reference documentation**. 2026c. Disponível em: https://docs.spring.io/spring-data/mongodb/reference/. Acesso em: 12 nov. 2026.

SPRINGDOC. **springdoc-openapi documentation**. 2026. Disponível em: https://springdoc.org/. Acesso em: 12 nov. 2026.

THYMELEAF. **Tutorial**: using Thymeleaf. 2026. Disponível em: https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html. Acesso em: 12 nov. 2026.